DAA – TOPIC 5

Experiment 1: Search an Element in a Sorted Array

• **Aim:** To search for an element in a sorted array using the Binary Search (Divide and Conquer) method.

• **Algorithm:**

1. Find the middle element of the sorted array.
2. Compare the middle element with the key.
3. If equal, return its position.
4. If the key is smaller, search the left half.
5. Otherwise, search the right half.
6. Repeat until the element is found or the search range becomes empty.

PROGRAM

Python Program

```
def binary_search(arr, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)

arr = [10, 20, 30, 40, 50]
key = 30
result = binary_search(arr, 0, len(arr) - 1, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

OUTPUT

Output

```
Element found at position 3
```

• **Conclusion:** Binary Search efficiently reduces the search space by half at each step. Its time complexity is O(log n).

Experiment 2: Find the Index of a Given Number

• **Aim:** To find the index of a target number in a sorted array using Binary Search with zero-based indexing.

• **Algorithm:**

1. Set the lower and upper bounds.
2. Find the middle index.
3. Compare the middle value with the target.
4. Search the left or right half based on the comparison.
5. Return the index if found; otherwise return -1.

PROGRAM

Python Program

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1

arr = [2, 4, 6, 8, 10, 12]
key = 8
print("Index =", binary_search(arr, key))
```

OUTPUT

Output

```
Index = 3
```

• **Conclusion:** The program returns the zero-based index of the target. Binary Search takes O(log n) time on a sorted array.

Experiment 3: Count the Number of Iterations in Binary Search

• **Aim:** To perform Binary Search on a sorted array and display the number of iterations required to find the element.

• **Algorithm:**

1. Initialize the lower and upper bounds.
2. Find the middle element in each iteration.
3. Compare it with the key.
4. Update the search range.
5. Count each middle-element comparison as one iteration.
6. Display the result and iteration count.

PROGRAM

Python Program

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    iterations = 0

    while low <= high:
        iterations += 1
        mid = (low + high) // 2
        if arr[mid] == key:
            return True, iterations
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return False, iterations

arr = [5, 10, 15, 20, 25, 30, 35]
key = 25
found, count = binary_search(arr, key)

print("Element found" if found else "Element not found")
print("Iterations =", count)
```

OUTPUT

Output

```
Element found
Iterations = 2
```

• **Conclusion:** The number of iterations grows logarithmically because the search interval is halved after each comparison. The complexity is O(log n).

Experiment 4: Find the First Occurrence of an Element

• **Aim:** To find the first occurrence of a given key in a sorted array containing duplicate elements using Binary Search.

• **Algorithm:**

1. Perform Binary Search on the sorted array.
2. When the key is found, store the index.
3. Continue searching in the left half for an earlier occurrence.
4. If no earlier occurrence exists, the stored index is the first occurrence.
5. Print the first occurrence or report that the element is absent.

PROGRAM

Python Program

```
def first_occurrence(arr, key):
    low, high = 0, len(arr) - 1
    result = -1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            result = mid
            high = mid - 1
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return result

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 2
result = first_occurrence(arr, key)

if result != -1:
    print("First occurrence at index", result)
else:
    print("Element not found")
```

OUTPUT

Output

```
First occurrence at index 1
```

• **Conclusion:** By continuing the search toward the left after finding the key, the program obtains the first occurrence efficiently in O(log n) time.

Experiment 5: Find the Last Occurrence of an Element

• **Aim:** To find the last occurrence of a given key in a sorted array containing duplicate elements using Binary Search.

• **Algorithm:**

1. Perform Binary Search on the sorted array.
2. When the key is found, store the index.
3. Continue searching in the right half for a later occurrence.
4. Keep updating the stored index whenever the key is found.
5. Display the last occurrence or report that the element is absent.

PROGRAM

Python Program

```
def last_occurrence(arr, key):
    low, high = 0, len(arr) - 1
    result = -1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            result = mid
            low = mid + 1
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return result

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 2
result = last_occurrence(arr, key)

if result != -1:
    print("Last occurrence at index", result)
else:
    print("Element not found")
```

OUTPUT

Output

```
Last occurrence at index 3
```

• **Conclusion:** The program searches toward the right after finding the key, allowing it to identify the last occurrence in O(log n) time.

Experiment 6: Highest and Lowest Temperature Recorded

• **Aim:** To find the minimum and maximum temperature recorded during a week using the Divide and Conquer approach.

• **Algorithm:**

1. Divide the temperature array into two halves.
2. Recursively find the minimum and maximum in each half.
3. Combine the two results.
4. Compare the two minimum values and two maximum values.
5. Display the overall minimum and maximum temperatures.

PROGRAM

Python Program

```
def min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    left_min, left_max = min_max(arr, low, mid)
    right_min, right_max = min_max(arr, mid + 1, high)

    return min(left_min, right_min), max(left_max, right_max)

temp = [32, 28, 35, 25, 31, 29, 37]
minimum, maximum = min_max(temp, 0, len(temp) - 1)

print("Minimum Temperature =", minimum)
print("Maximum Temperature =", maximum)
```

OUTPUT

Output

```
Minimum Temperature = 25
Maximum Temperature = 37
```

• **Conclusion:** Divide and Conquer divides the data into smaller parts and combines their minimum and maximum values. The time complexity is O(n).

Experiment 7: Best and Worst Student Marks

• **Aim:** To identify the highest and lowest marks scored by students using Divide and Conquer.

• **Algorithm:**

1. Divide the marks list into two halves.
2. Find the minimum and maximum of each half recursively.
3. Combine the results from both halves.
4. Select the smaller minimum and larger maximum.
5. Display the minimum and maximum marks.

PROGRAM

Python Program

```
def min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    min1, max1 = min_max(arr, low, mid)
    min2, max2 = min_max(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)

marks = [78, 92, 65, 88, 95, 72, 81, 69]
minimum, maximum = min_max(marks, 0, len(marks) - 1)

print("Minimum Mark =", minimum)
print("Maximum Mark =", maximum)
```

OUTPUT

Output

```
Minimum Mark = 65
Maximum Mark = 95
```

• **Conclusion:** The Divide and Conquer method correctly identifies the best and worst marks. Its overall time complexity is O(n).

Experiment 8: Stock Price Analysis

• **Aim:** To determine the minimum and maximum stock prices recorded during a trading week using Divide and Conquer.

• **Algorithm:**

1. Divide the stock price list into two parts.
2. Recursively determine the minimum and maximum in each part.
3. Compare the two minimum values.
4. Compare the two maximum values.
5. Display the minimum and maximum stock prices.

PROGRAM

Python Program

```
def min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    a, b = min_max(arr, low, mid)
    c, d = min_max(arr, mid + 1, high)

    return min(a, c), max(b, d)

prices = [450, 520, 480, 610, 430, 590]
minimum, maximum = min_max(prices, 0, len(prices) - 1)

print("Minimum Stock Price =", minimum)
print("Maximum Stock Price =", maximum)
```

OUTPUT

Output

```
Minimum Stock Price = 430
Maximum Stock Price = 610
```

• **Conclusion:** The algorithm efficiently finds both extreme stock prices by recursively dividing the input. Its time complexity is O(n).

Experiment 9: Fastest and Slowest Race Times

• **Aim:** To find the fastest and slowest race completion times using the Divide and Conquer method.

• **Algorithm:**

1. Divide the race-time list into two halves.
2. Recursively find the fastest and slowest time in each half.
3. Compare the minimum times from both halves.
4. Compare the maximum times from both halves.
5. Display the fastest and slowest times.

PROGRAM

Python Program

```
def min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    min1, max1 = min_max(arr, low, mid)
    min2, max2 = min_max(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)

times = [14, 12, 18, 11, 15, 13, 17, 16, 10]
fastest, slowest = min_max(times, 0, len(times) - 1)

print("Fastest Time =", fastest)
print("Slowest Time =", slowest)
```

OUTPUT

Output

```
Fastest Time = 10
Slowest Time = 18
```

• **Conclusion:** The program successfully finds the fastest and slowest race times using recursive division. The time complexity is O(n).

Experiment 10: Largest and Smallest Building Heights

• **Aim:** To determine the tallest and shortest buildings in a city block using Divide and Conquer.

• **Algorithm:**

1. Divide the building-height list into two halves.
2. Find the minimum and maximum height in each half recursively.
3. Combine the results from both halves.
4. Choose the smallest height and largest height.
5. Display the minimum and maximum building heights.

PROGRAM

Python Program

```
def min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    min1, max1 = min_max(arr, low, mid)
    min2, max2 = min_max(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)

heights = [120, 150, 98, 175, 140, 165, 110, 190, 130, 145]
minimum, maximum = min_max(heights, 0, len(heights) - 1)

print("Minimum Height =", minimum)
print("Maximum Height =", maximum)
```

OUTPUT

Output

```
Minimum Height = 98
Maximum Height = 190
```

• **Conclusion:** The Divide and Conquer approach correctly identifies the shortest and tallest buildings. The time complexity is O(n).